

Neural Network Fundamentals with PyTorch

Running example: learning the inverse dynamics of a 2-link robot arm

MLP architecture · backpropagation · PyTorch tensors · autograd

Dataset and DataLoader · building and training a model · saving and deployment

Companion notebook: `mlpi26_Lect07_nbsup1.ipynb`

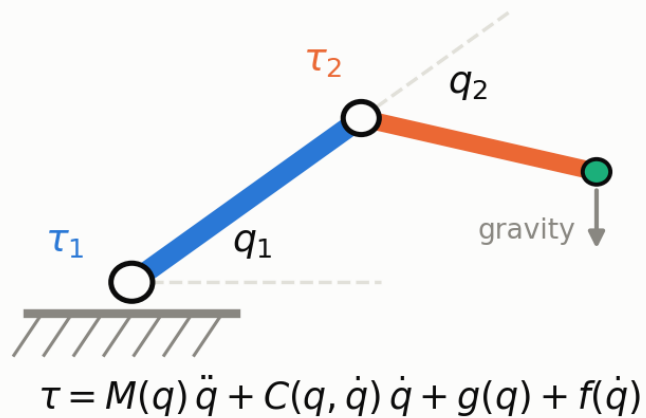
9,000 logged samples · 4,738 parameters · trains in about 10 seconds on a laptop CPU

THE PROBLEM

What torque should each motor produce?

A robot controller answers this a thousand times a second. For a rigid two-link arm you can derive the answer; for a real arm with belts, harmonic drives and joint flexibility you cannot — so you log the motion and learn the map instead.

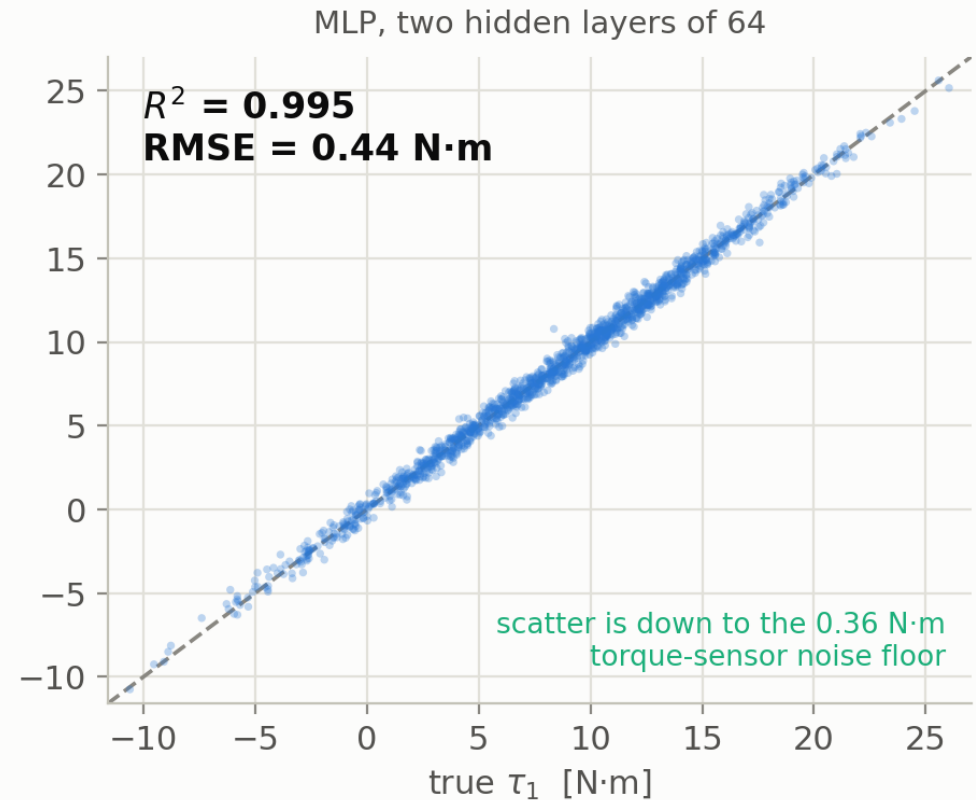
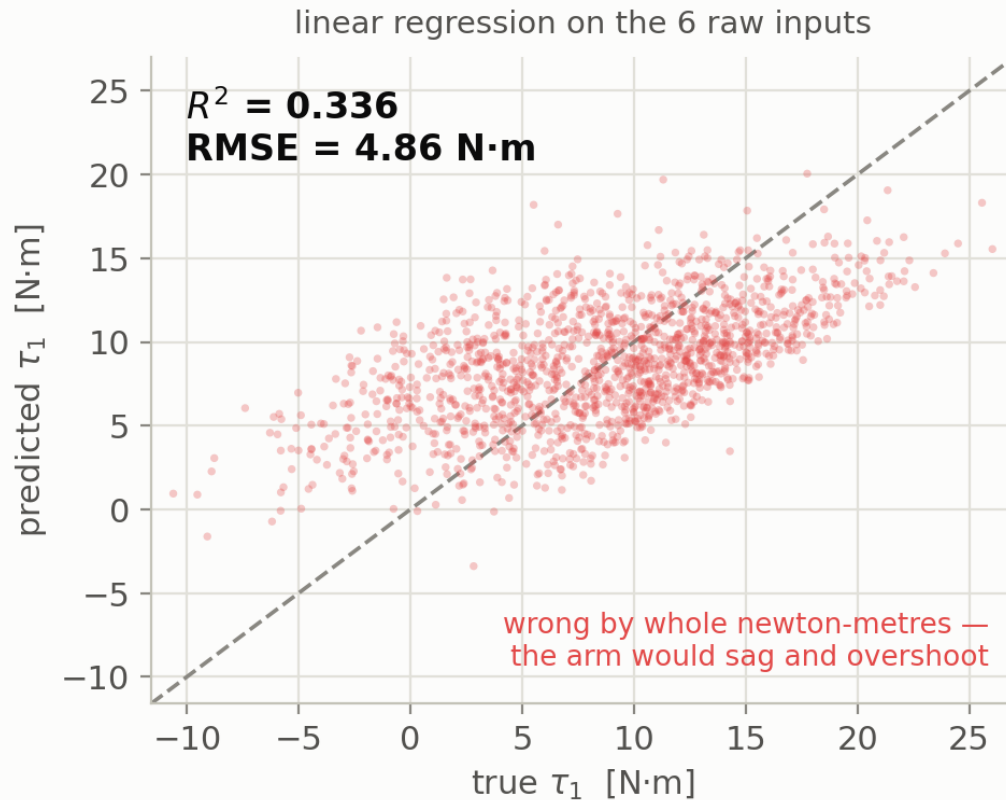
2-DOF planar arm · what torque holds this pose and this motion?



Six inputs — joint angles, velocities, desired accelerations. Two outputs — shoulder and elbow torque. 9,000 logged samples, split 6,000 / 1,500 / 1,500 into train, validation and test.

A straight line cannot do this

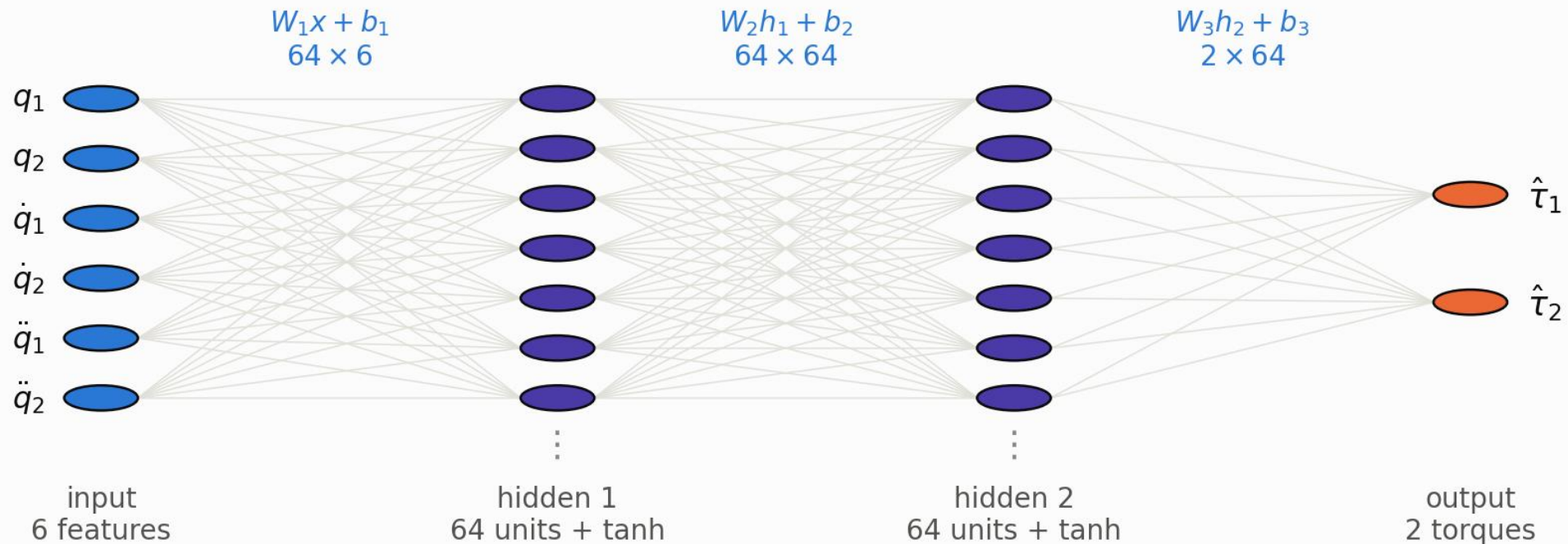
Always fit the dumb model first. Linear regression on the six raw inputs is the honest benchmark that tells you whether a neural network was needed at all.



Linear: RMSE 4.86 N·m, R^2 0.336. MLP: RMSE 0.44 N·m, R^2 0.995 — an 11× reduction, and only 1.23× above the 0.36 N·m sensor-noise floor.

The multilayer perceptron

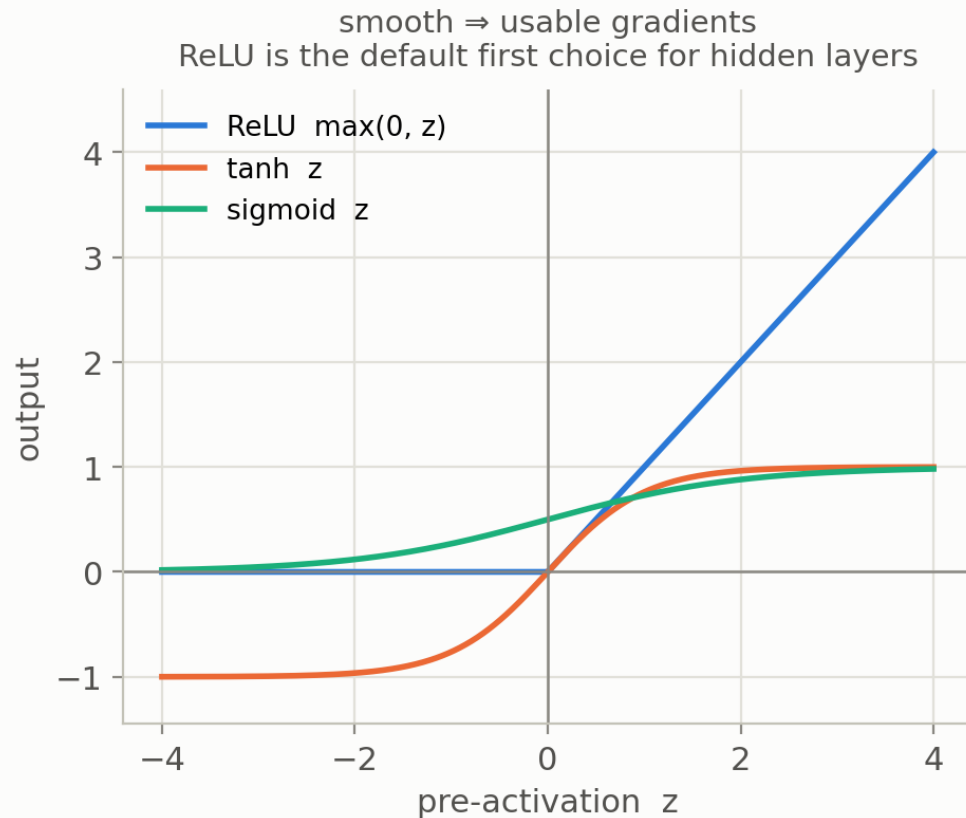
A stack of affine maps, each followed by a nonlinearity. Every arrow is a weight, every weight is learned, and the shapes are fixed the moment you choose the widths.



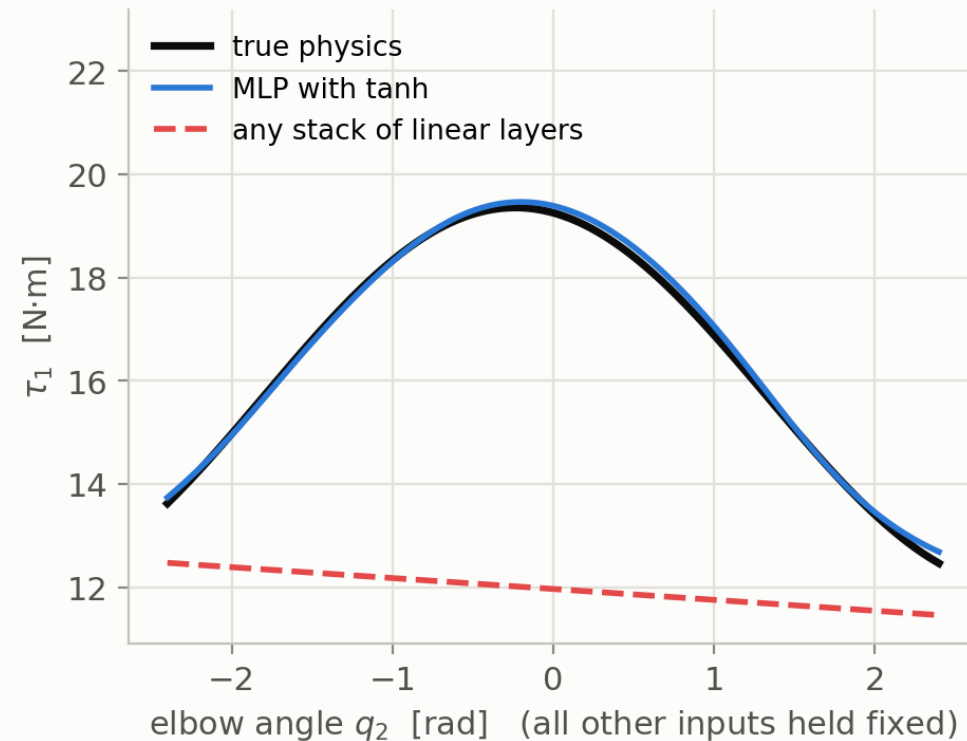
Two hidden layers of 64 units: 4,738 parameters for 6,000 training samples. Width and depth are design choices — start small, grow only while the validation loss keeps improving.

Where the nonlinearity comes from

The activation function is not a detail — it is the only reason a deep network can do anything a single matrix cannot.



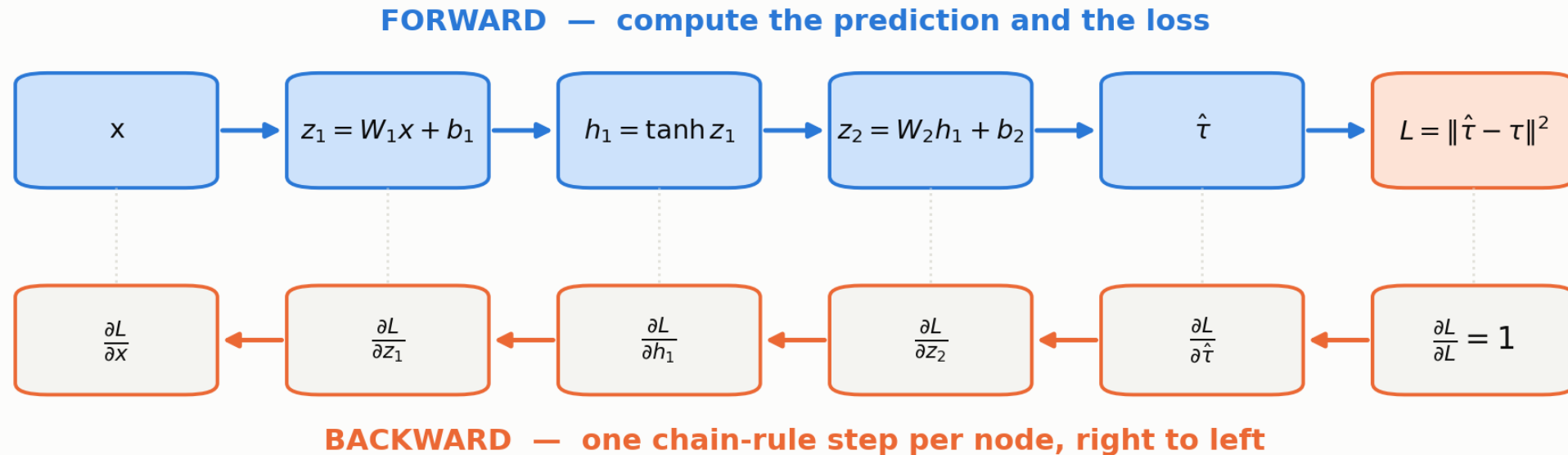
remove the activation and the whole network collapses to one line



Two stacked linear layers are $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + b$ — still one linear map. Ten layers, still one linear map. Remove the activations and the network collapses to the red line.

Backpropagation is the chain rule, node by node

Forward: compute the prediction and the loss. Backward: hand each node the gradient arriving from its right, multiply by its own local derivative, pass it left.



each node needs only its own local derivative and the gradient handed to it from the right $\rightarrow$ cost of the backward pass $\approx$ cost of the forward pass

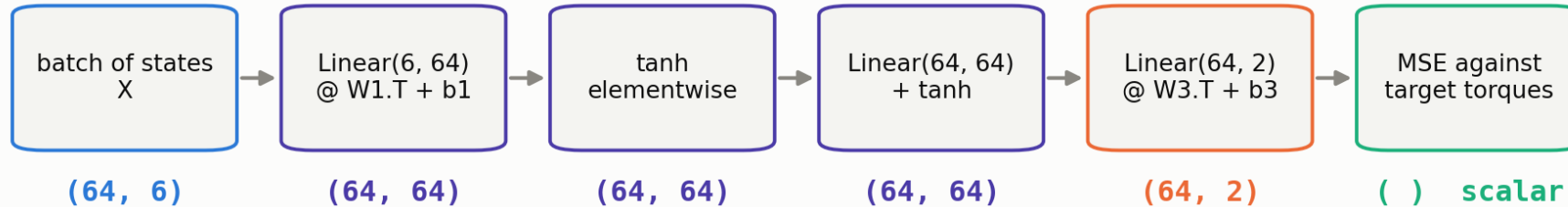
In the notebook we code these five lines by hand for a one-hidden-layer network and compare with autograd: maximum disagreement 1.5×10^{-8} — the same chain rule, written twice.

Tensors: shape, dtype, device, gradient

A tensor is a NumPy array that also knows where it lives and how to differentiate itself. The first axis is always the batch.

every arrow is a tensor; the first axis is always the batch

a tensor = numbers + shape + dtype + device + (optionally) a gradient



```
torch.float32 on cpu · x.shape (64, 6) · x.dtype · x.device · x.requires_grad
```

get a shape wrong and PyTorch tells you immediately — read the two numbers in the error message, they are the matrix dimensions

```

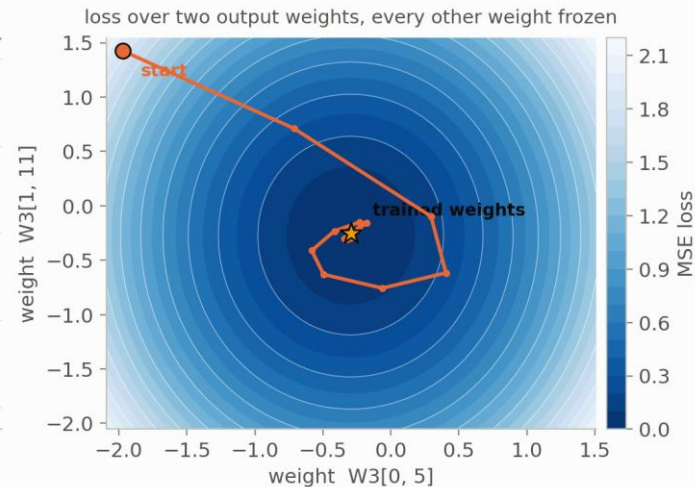
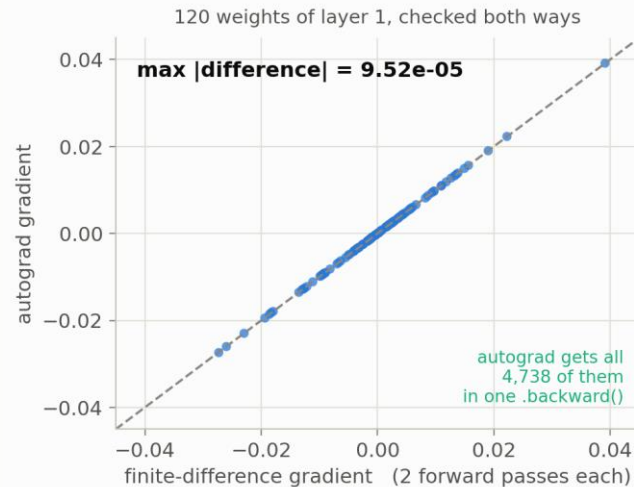
X = torch.tensor(states, dtype=torch.float32, device=device) # (6000, 6)
X.shape, X.dtype, X.device, X.requires_grad
batch = X[:64] # (64, 6) slicing is NumPy's
single = X[0].unsqueeze(0) # (6,) -> (1, 6): a batch of one
# RuntimeError: mat1 and mat2 shapes cannot be multiplied (64x8 and 6x64)
# -> 8 columns arriving at a layer built for 6. Print both .shape values.
  
```

Autograd: every gradient from one call

PyTorch records the operations you perform into a graph, then walks it backwards. You never differentiate anything by hand again — but you do have to clear the gradients yourself.

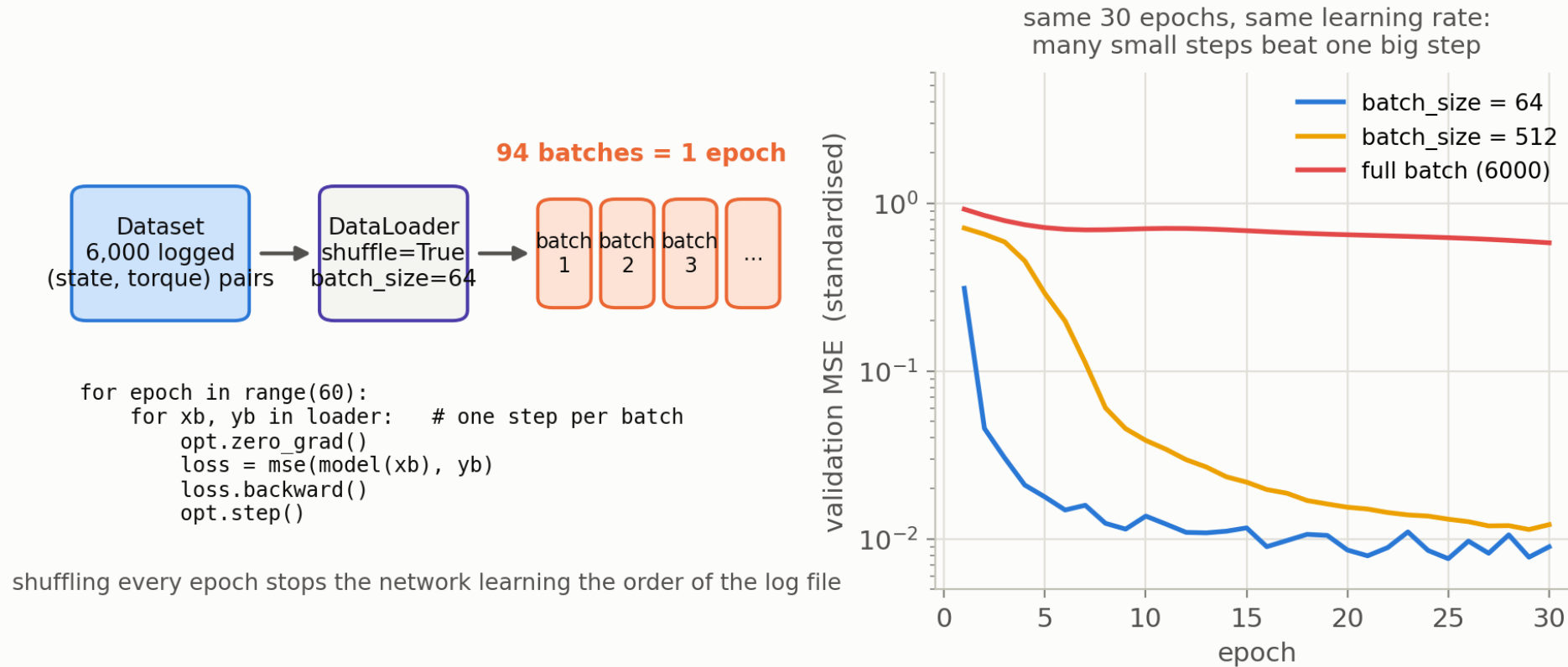
```
w = torch.tensor(4.0, requires_grad=True)
f = 3*w**2 + 2*w ; f.backward()      # w.grad == 26.0 == 6w + 2

optimizer.zero_grad()              # gradients ACCUMULATE – clear them first
loss.backward()                     # fills .grad on all 4,738 parameters
optimizer.step()                    # uses them
with torch.no_grad():               # evaluation / inference: build no graph
```



Dataset and DataLoader: feeding the network

A Dataset answers 'how many samples' and 'give me sample i'. A DataLoader turns that into shuffled mini-batches — one gradient step each.



6,000 samples at batch_size 64 = 94 gradient steps per epoch. One full-batch step per epoch leaves the loss at 0.58 after 30 epochs; 94 small ones reach 0.009.

nn.Module: `__init__` declares, forward connects

Declare the layers once, then say how data flows through them. You call `model(x)`, never `model.forward(x)`.

```
class ArmMLP(nn.Module):
    def __init__(self, n_in=6, n_hidden=64, n_out=2):
        super().__init__()          # always first
        self.fc1 = nn.Linear(n_in, n_hidden)
        self.fc2 = nn.Linear(n_hidden, n_hidden)
        self.fc3 = nn.Linear(n_hidden, n_out)
        self.act = nn.Tanh()

    def forward(self, x):
        h = self.act(self.fc1(x))
        h = self.act(self.fc2(h))
        return self.fc3(h)          # NO activation on a regression output
```

nn.Sequential

A straight pipeline and nothing else. Perfect for an MLP, and one line shorter.

Subclass nn.Module

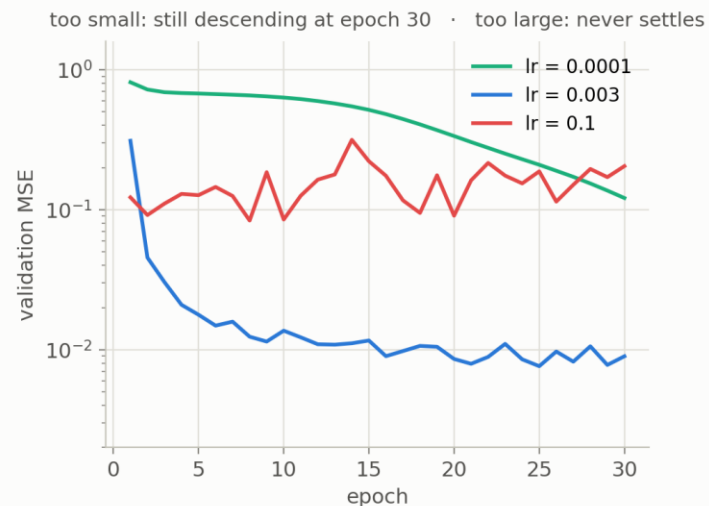
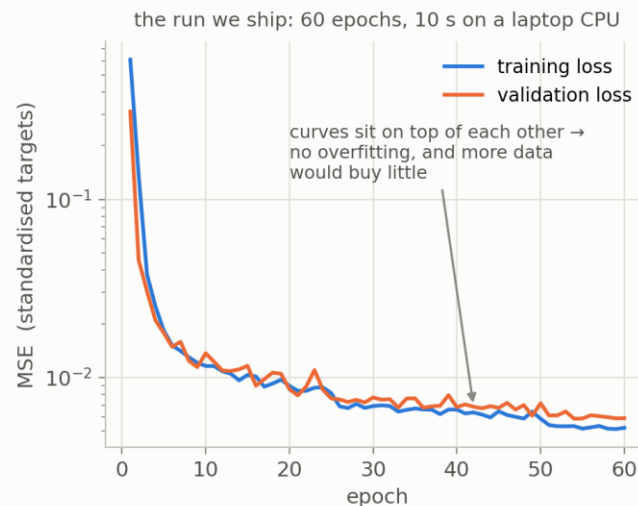
Anything with branching: skip connections, two inputs, a physics term added to the network output.

Torques run from -12 to $+29$ N·m — a tanh or sigmoid on the output layer would clamp the network to ± 1 and it could never fit.

The training loop is four lines

Everything else is bookkeeping. These four lines, in this order, are the same in every PyTorch project you will ever write.

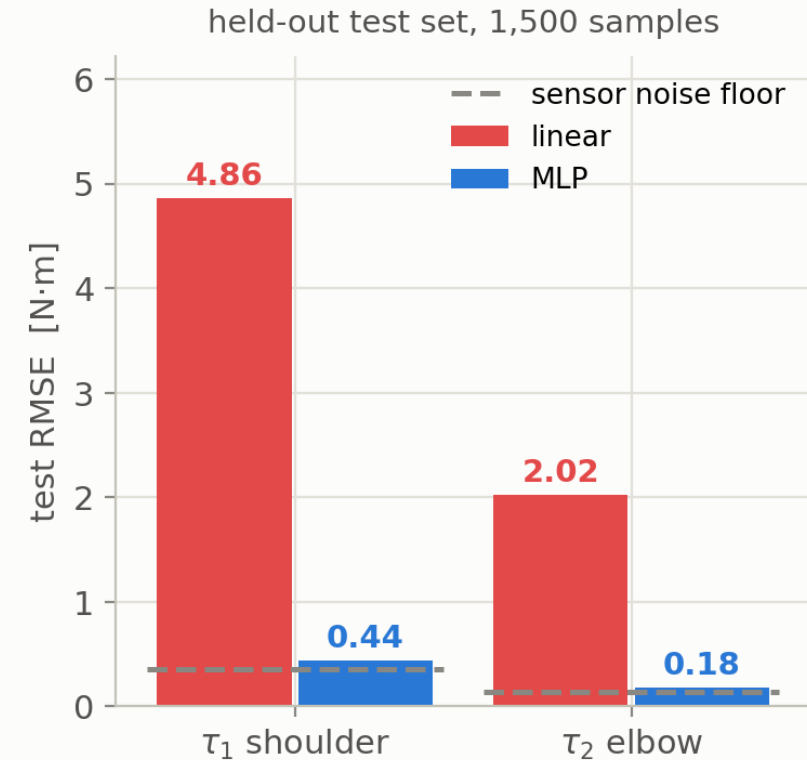
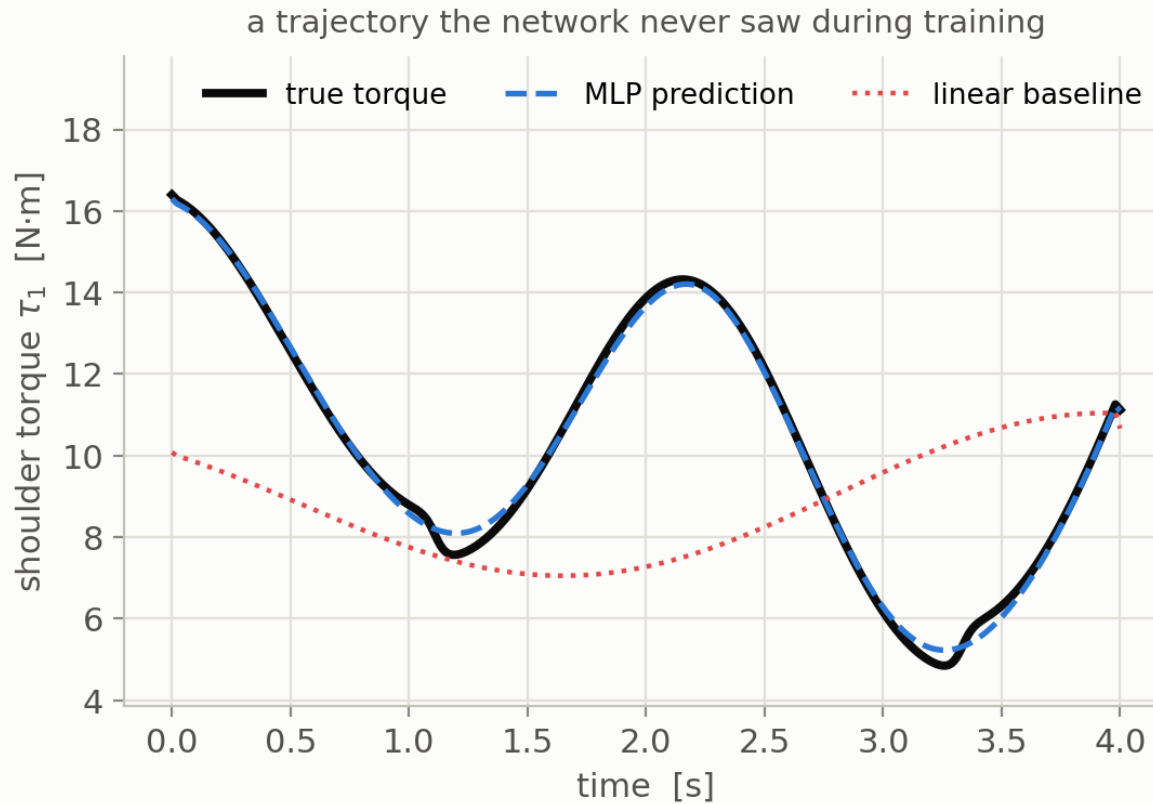
```
for epoch in range(60):
    model.train()
    for xb, yb in train_loader:
        optimizer.zero_grad()           # 1 clear last step's gradients
        loss = criterion(model(xb), yb) # 2 forward
        loss.backward()                 # 3 backward
        optimizer.step()                # 4 update every parameter
    val_loss = evaluate(model, val_loader) # under torch.no_grad()
```



EVALUATION

Does it work? Look at the test set once

Report the error in the engineering unit, next to the baseline you have to beat and the noise floor you cannot beat.

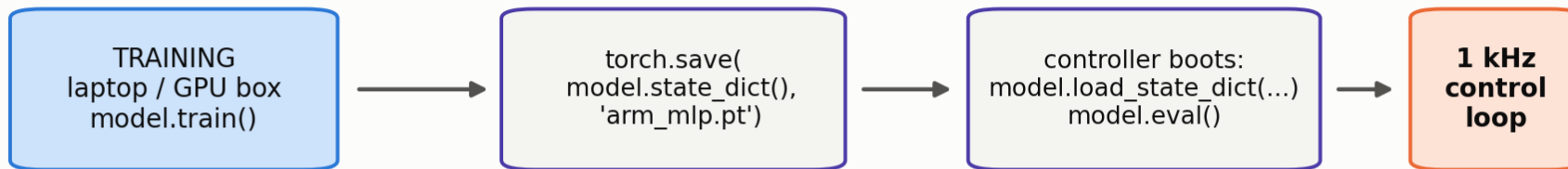


Test RMSE 0.44 and 0.18 N·m against 4.86 and 2.02 for the linear model — and the torque sensor itself is worth 0.36 N·m. There is almost nothing left to win.

Save the weights, not the model object

`torch.save(model)` pickles your class definition with it, so the file breaks the moment you rename a file. Save the `state_dict` — and the normalisation constants.

the weights are the deliverable — not the training script



state_dict on disk: 21 KB

one forward pass: tens of μ s on CPU

with `torch.no_grad()` + `model.eval()` — no graph is built, dropout and batch-norm switch to inference behaviour

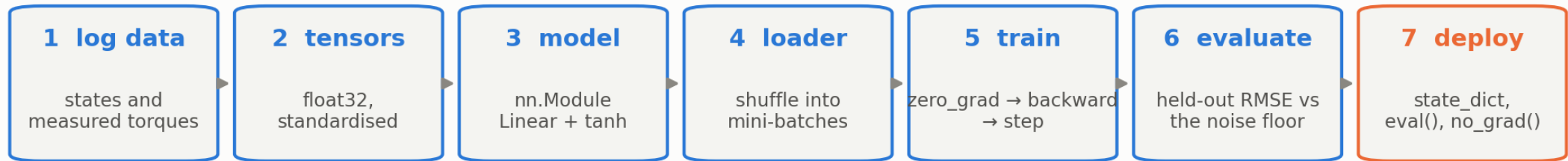
well under 5 % of the 1000 μ s budget of a 1 kHz control loop

```
torch.save({'state_dict': model.state_dict(), 'arch': {...},
          'x_mu': x_mu, 'x_sd': x_sd, 'y_mu': y_mu, 'y_sd': y_sd}, 'arm_mlp.pt')
```

```
model = ArmMLP(**ckpt['arch']); model.load_state_dict(ckpt['state_dict'])
model.eval()                    # inference behaviour
with torch.no_grad(): tau = model(state)    # no graph, no gradients
```

Seven steps, and they never change

the loop you will repeat for every network you ever build



steps 2-5 are the same three lines of PyTorch every time; only the model and the data change

1. Split first, then standardise with training statistics only — every leak flatters your numbers.
2. The first axis is the batch. Print `.shape` the moment anything surprises you.
3. `zero_grad` → `forward` → `backward` → `step`. In that order, every time.
4. No activation on a regression output layer.
5. Watch training and validation loss together: apart means overfitting, both high means underfitting.
6. Beat a dumb baseline, then stop at the noise floor. Test set once, at the end.
7. Ship the `state_dict` plus the normalisation constants; load with `eval()` and `no_grad()`.